Universidade de Brasília
Instituto de Ciências Exatas – Departamento de Ciência da Computação

Prova-T (peso 2,5) CIC0104 Software Básico, semestre: 21/2 – Brasília,

[redacted]

A prova é individual e sem consulta. Se for descoberta cola será atribuída nota zero para quem colou e para quem deu cola. Nas questões objetivas preencha a segunda coluna com **F (falso)**, **V (verdade)** ou **NS** (**não sei**). Duas respostas erradas anulam uma certa. Resposta em **branco** conta como resposta **errada**. Justifique as questões respondidas como falso. Cada resposta corresponde a 0,32. Mantenha o formato de tabela desta prova, não mude esse layout!

Parte 1 – questões objetivas (7,0)

1. A arquitetura de von Neumann é caracterizada por CPU, memória e periféricos, sendo a CPU composta por unidade lógica e aritmética, unidade de controle, registradores e contador de programas – PC. Na memória RAM são armazenados dados e programas.	F
2. A execução é rápida na arquitetura de von Neumann porque dados e instruções são armazenados em blocos de memória adjacentes devido ao fenômeno de localidade de referência.	V
3. A função básica do ligador é ligar os módulos e resolver as referências externas, seguindo as seguintes etapas: ler e relocar de forma estática o módulo objeto P; atualizar a tabela global de símbolos com os símbolos internos de P que podem ser referenciados em outros módulos; concatenar P aos módulos anteriores; e resolver as referências externas em P.	V
4. A redução do gargalo de von Neumann entre CPU e periféricos de I/O pode ser obtida com o uso de memória **cache** e processadores **pipelining** com múltiplos estágios.	F
5. Ao se ligar um computador, pressionando o botão de **power on**, um carregador absoluto armazenado em memória volátil é ativado para iniciar o processo de carga do sistema operacional.	F
6. As otimizações algorítmicas de código que podem ser realizadas na síntese do código são de tempo de execução ou de alocação de memória	F/NS
7. Carregador de relocação estática lê o tamanho do programa a carregar; solicita ao sistema operacional (S.O.) alocar um bloco contíguo de memória com tamanho suficiente para conter o programa; carrega o programa a partir do endereço dado pelo S.O., reloca os endereços relocáveis; e inicializa o PC com o **entrypoint** do programa.	V
8. Código absoluto foi utilizado no início da computação com sistemas monousuário e não é mais utilizado devido a que os sistemas atuais são multiprogramados ou multiprocessados.	F
9. Do ponto de vista lógico, em geral um montador é estruturado em um pré-processamento (retira comentários, expande macro e faz a montagem condicional), passo 1 (análises léxica, sintática e semântica) e passo 2 (otimização e geração de código absoluto).	F
10. Em um S.O. do tipo **batch** a melhor gerência de memória real é paginação-segmentação.	F
11. Em um S.O. do tipo time **sharing** como as transações são CPU intensiva, a melhor gerência de memória real é de partições.	F
12. Gerência de memória baseada em páginas minimiza a fragmentação interna, favorece a proteção de código e a fragmentação externa.	F
13. Ligadores são dos tipos: **Linkage Editor** (gera um arquivo binário que pode ser reusado) ou **Linking Loader**/**Binder** (carrega o código gerado na memória sem gerar o arquivo que possa ser reusado), ou **carregador de função** de biblioteca de ligação dinâmica (tipo DLL).	V
14. Na arquitetura de von Neumann o processamento é sequencial e segue o ciclo: busca – decodificação – busca de operandos – execução. O carregador carrega o programa na memória, atribui a PC o valor do **entry point** (endereço de uma instrução) e inicia o ciclo.	F
15. O advento do código relocável permitiu o uso de bibliotecas de código objeto como é o caso do **bytecode** Java que roda em qualquer computador com uma JVM Java instalada.	NS

16. Para maximizar o **throughput** e evitar **deadlock** um processo só pode entrar na fila de prontos se já estiver com todos os recursos que necessitar.	F
17. Região crítica é o trecho do código que objetiva garantir a exclusão mútua de processos.	V
18. Se um processo de posse da CPU solicitar acesso a uma página não disponível na memória será criado um programa de canal para carregar a página em uma **page frame** e o processo ficará bloqueado.	V
19. Sistemas de escalonamento procuram maximizar o **throughput** e minimizar o **response time** e o **turnaround time.** Por isso um bom escalonador deve ser não preemptivo.	F
20. Um escalonamento de processos adequado para um S.O. multiprocessado é o **round-robin** pois permite evitar **deadlock.**	F
21. Um ligador, a partir de arquivos de módulos objetos relocáveis, resolve as referências externas e gera um arquivo contendo os códigos objetos ligados, e com a indicação dos endereços relocáveis. Esse arquivo, união dos códigos objetos, não é executável.	F
22. Um montador é um programa que converte um programa em código Assembly em um código objeto relocável (código de máquina), se o programa Assembly estiver correto, e cria um arquivo binário com código de máquina relocável, contendo uma tabela (ou lista) de endereços relocáveis e uma lista de símbolos não resolvidos pelo montador.	V

Justificativas para as questões respondidas como falso

1. A arquitetura de von Neumann é caracterizada por CPU, memória, periféricos e barramentos. A memória armazena dados e programas.
4. A solução apresentada é para a redução do gargalo de von Newumann entre CPU e memória.
5. O carregador absoluto é armazenado em memória não volátil.
6. A otimização algorítimica do código pode ser realizadas pelo montador.
8. Dependendo do sistema, o código absoluto ainda pode ser utilizado atualmente.
9. O montador gera um código objeto e não um código absoluto.
10. Paginação-segmentação é uma abordagem de gerência de memória virtual.
14. A cada ciclo de busca, PC é incrementado.
19. Um bom escalonador pode ser preemptivo ou não preemptivo.
21. O arquvio gerado pelo ligador é executável, alguns ligadores apenas nâo geram esse arquivo.
12. O modo de gerencia descrito é o baseado em segmentação.
16. A solução descrita não maximiza o throughput.
20. Round-robin evita starvation.
11. Sistema operacional do tipo batch.

Parte 2 - questões discursivas

1. Caracterize o que é (0,5) e o que causa localidade de referência (0,5).

(1,0)

Localidade de referência é a tendência do processador acessar várias vezes as mesmas áreas de memória em um curto período de tempo. Pode ser causada pelo acesso ao mesmo dado várias vezes, ou pelo acesso à vários dados em endereços muito próximos.

2. **Quais os tipos de carregadores existentes? (0,5) Cite o instante em que os endereços são relocados (0,25) e o que é carregado em cada um dos tipos de carregador que citar (0,25). (1,0)**

Carregador Absoluto: Inicia a carga do sistema operacional

Carregador de Relocação: Carrega na memória RAM o programa executável contido em disco

3. **Apresente um exemplo e explique como é realizado o mapeamento de um endereço lógico em um endereço real em um ambiente paginado. (1,0)**

Um endereço real é a soma do endereço lógico com um registrador base. Em um ambiente paginado, além dessa soma, é preciso acessar uma tabela de mapeamento de páginas, que contém uma entrada para cada página física do programa, para compor o endereço real final.

4. *Coloque-se na posição de projetista de S.O. Indique qual é a aplicação característica (**batch**, **real time**, **time sharing**, rede ou distribuído), forma de carregamento de programas na memória (contíguos ou não), tipo (monoprogramado, multiprogramado ou multiprocessado), e estrutura (hierárquica, máquina virtual, multiprocessado, distribuído ou orientado a objetos). Apresente e justifique a **política de processos adequada** (0,5) indicando como ela evita **deadlock** (0,5) e **starvation** (0,5), e a **política de gerência de memória real**. (0,5) (2,0)*

Time sharing, alocação de memória contígua, multiprogramado.

Gerencia de memória real: Swapping

Política de processos: Time-slice

No swapping é ideal ter alocação de memória contígua e um sistema multiplogramado, funciona a base de time-slice com troca de processos via interrupção. Com isso, a aplicação caracterísa ideal é o time sharing.

Para evitar deadlock, os processos devem liberar os recursos quando não for mais necessário.

Para evitar starvation, o swapping escalona os processos por prioridade.